MongoDB — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction to NoSQL and MongoDB

02 Installation, Atlas and mongosh

03 Databases, Collections and Documents

04 Insert Documents

05 Find and Query Operators

06 Update and Delete

Intermediate (6 topics)

07 Data Modeling: Embedding vs Referencing

08 Indexes

09 Aggregation Pipeline

10 Schema Validation

11 Using MongoDB from Node.js

12 Array and Embedded Document Queries

Advanced (6 topics)

13 Transactions

14 Replication and Replica Sets

15 Sharding

16 Security

17 Performance Tuning

18 Change Streams and Time Series

Master (4 topics)

19	Atlas Search and Vector Search
20	Backup, Restore and Monitoring
21	Production Best Practices
22	Real Project: Blog / E-commerce Backend

Beginner Level

01. Introduction to NoSQL and MongoDB

DEFINITION

MongoDB is a document database storing flexible JSON-like (BSON) documents in collections instead of rows in tables.

SYNTAX

```
mongosh
```

EXAMPLE

```
mongosh
show dbs
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A cupboard where each family's papers are kept in their own folder of any shape; MongoDB stores flexible JSON-like documents.

தமிழ்

ஒவ்வொரு குடும்பத்தின் ஆவணங்களும் எந்த வடிவிலும் தனிக் கோப்பில் இருக்கும் அலமாரி போல MongoDB நெகிழ்வான JSON போன்ற ஆவணங்களைச் சேமிக்கிறது.

PROS

- Flexible schema
- Easy horizontal scaling
- Developer friendly JSON
- Rich query and aggregation

CONS

- No joins like SQL (use `$lookup`)
- Data duplication needs care
- Less strict integrity by default

ADVANCED / NEW FEATURES

- MongoDB 8.0 performance gains
- Queryable Encryption
- Atlas Vector Search
- Time series collections

02. Installation, Atlas and mongosh

DEFINITION

Install MongoDB Community or use free MongoDB Atlas in the cloud; connect with `mongosh` or Compass.

SYNTAX

```
mongosh "mongodb+srv://..."
```

EXAMPLE

```
mongosh "mongodb+srv://cluster.example.mongodb.net/test"
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Using a rented locker (Atlas) in the cloud instead of building your own storeroom; Atlas gives a ready MongoDB cluster.

தமிழ்

சொந்தக் கிடங்கு கட்டாமல் மேகத்தில் வாடகை லாக்கர் (Atlas) பயன்படுத்துவது போல Atlas தயாரான MongoDB cluster தருகிறது.

PROS

- Free tier
- Managed backups and scaling

CONS

- Cloud costs grow
- Network latency

ADVANCED / NEW FEATURES

- Atlas Search
- Atlas Vector Search
- Serverless instances

03. Databases, Collections and Documents

DEFINITION

A database holds collections; a collection holds documents with an `_id` primary key.

SYNTAX

```
use db
db.createCollection('c')
```

EXAMPLE

```
use shop
db.createCollection('products')
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A cupboard (database) with shelves (collections) holding folders (documents).

தமிழ்

அலமாரி (database), அதில் தட்டுகள் (collections), அவற்றில் கோப்புகள் (documents).

PROS

- Simple hierarchy
- Collections created on first insert

CONS

- No enforced schema unless you add validation

ADVANCED / NEW FEATURES

- Schema validation rules
- Clustered collections

04. Insert Documents

DEFINITION

`insertOne` and `insertMany`; documents may have nested objects and arrays.

SYNTAX

```
db.c.insertOne({ ... })
```

EXAMPLE

```
db.products.insertOne({ name: 'Laptop', price: 55000, tags: ['tech'] })
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Adding a new customer card of any shape into the file box; each document may have different fields.

தமிழ்

எந்த வடிவிலும் புதிய வாடிக்கையாளர் அட்டையை பெட்டியில் சேர்ப்பது போல ஒவ்வொரு document-க்கும் வேறு புலங்கள் இருக்கலாம்.

PROS

- Fast writes
- Nested data in one document

CONS

- Document size limit 16 MB

ADVANCED / NEW FEATURES

- Bulk write improvements in 8.0

05. Find and Query Operators

DEFINITION

`find`, filters with `$gt`, `$lt`, `$in`, `$and`, `$or`, `$regex`; projection, sort, limit.

SYNTAX

```
db.c.find({ price: { $gt: 100 } })
```

EXAMPLE

```
db.products.find({ price: { $gt: 1000 } }, { name: 1 }).sort({ price: -1 }).limit(5)
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Asking the clerk to bring all folders where price is above 100; queries filter documents by conditions.

தமிழ்

விலை 100-க்கு மேல் உள்ள கோப்புகளை எழுத்தரிடம் கேட்பது போல queries நிபந்தனைகளால் ஆவணங்களை வடிகட்டுகின்றன.

PROS

- Rich operators
- Dot notation for nested data

CONS

- Unindexed queries scan whole collections

ADVANCED / NEW FEATURES

- Improved query planner
- Queryable encryption range queries

06. Update and Delete

DEFINITION

`updateOne`, `updateMany` with `$set`, `$inc`, `$push`, `$pull`; `deleteOne`, `deleteMany`.

SYNTAX

```
db.c.updateOne(filter, { $set: { a: 1 } })
```

EXAMPLE

```
db.products.updateOne({ name: 'Laptop' }, { $inc: { price: -500 } })
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Correcting one field on a customer card without rewriting the card; update operators change parts of a document.

தமிழ்

அட்டையை மீண்டும் எழுதாமல் ஒரு புலத்தை மட்டும் திருத்துவது போல update operators ஆவணத்தின் பகுதிகளை மாற்றுகின்றன.

PROS

- Atomic single-document updates
- Array operators

CONS

- Large updates can be slow

ADVANCED / NEW FEATURES

- Update with aggregation pipeline

Intermediate Level

07. Data Modeling: Embedding vs Referencing

DEFINITION

Embed data that is read together; reference data that is large, shared or grows without bound.

SYNTAX

```
{ order: { items: [...] } }
```

EXAMPLE

```
{ _id: 1, name: 'Arun', addresses: [{ city: 'Chennai' }] }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Writing the items inside the bill itself (embedding) versus keeping a reference number to a separate item list; modelling decides which.

தமிழ்

பில்லிலேயே பொருட்களை எழுதுவது (embedding) அல்லது தனிப் பட்டியலுக்கு குறிப்பு எண் வைப்பது; modelling இதைத் தீர்மானிக்கிறது.

PROS

- Fast reads with embedding
- Flexible

CONS

- Unbounded arrays hurt
- Duplication when embedding

ADVANCED / NEW FEATURES

- Schema design patterns (bucket, outlier)
- Atlas schema suggestions

08. Indexes

DEFINITION

Single, compound, text, TTL and unique indexes; use `explain()` to verify.

SYNTAX

```
db.c.createIndex({ field: 1 })
```

EXAMPLE

```
db.users.createIndex({ email: 1 }, { unique: true })
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A book's index so the clerk finds a folder without checking every one; indexes speed up queries.

தமிழ்

ஒவ்வொரு கோப்பையும் பார்க்காமல் எழுத்தர் கண்டுபிடிக்க உதவும் புத்தக அட்டவணை போல indexes queries-ஐ வேகப்படுத்துகின்றன.

PROS

- Big speed-ups
- Unique, TTL, text, compound indexes

CONS

- Extra storage/slower writes

ADVANCED / NEW FEATURES

- Hidden indexes
- Wildcard indexes

09. Aggregation Pipeline

DEFINITION

Process data in stages: `$match`, `$group`, `$project`, `$sort`, `$lookup`, `$unwind`.

SYNTAX

```
db.c.aggregate([{ $match }, { $group }])
```

EXAMPLE

```
db.orders.aggregate([
  { $match: { status: 'paid' } },
  { $group: { _id: '$city', total: { $sum: '$amount' } } }
])
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A factory conveyor: items pass through stations (filter, group, sort) to produce a report.

தமிழ்

ஒரு தொழிற்சாலை கன்வேயர்: பொருட்கள் நிலையங்கள் (filter, group, sort) வழியாகச் சென்று அறிக்கையாகும்.

PROS

- Powerful analytics
- Server-side processing

CONS

- Pipelines can be complex
- Memory limits per stage

ADVANCED / NEW FEATURES

- `$lookup` improvements
- Atlas SQL interface

10. Schema Validation

DEFINITION

Enforce structure with JSON Schema validators on collections.

SYNTAX

```
db.createCollection('c', { validator: { $jsonSchema: ... } })
```

EXAMPLE

```
db.createCollection('users', { validator: { $jsonSchema: { required: ['email'] } } })
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A form template that rejects entries missing required boxes even in a flexible file box; validation adds rules.

தமிழ்

நெகிழ்வான பெட்டியிலும் கட்டாயக் கட்டங்கள் இல்லாத பதிவுகளை நிராகரிக்கும் படிவ மாதிரி போல validation விதிகளைச் சேர்க்கிறது.

PROS

- Keeps data clean
- Gradual strictness

CONS

- Existing bad data needs cleanup

ADVANCED / NEW FEATURES

- Validation levels and actions

11. Using MongoDB from Node.js

DEFINITION

Official driver or Mongoose ODM with schemas and models.

SYNTAX

```
await User.create({ name })
```

EXAMPLE

```
const User = mongoose.model('User', new Schema({ name: String }));
await User.create({ name: 'Priya' });
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Your shop software talking to the cupboard through a helper (Mongoose) that knows the folder format.

தமிழ்

கோப்பு வடிவம் அறிந்த உதவியாளர் (Mongoose) வழியாக உங்கள் கடை மென்பொருள் அலமாரியுடன் பேசுவது போல.

PROS

- Schemas and validation in code
- Rich ecosystem

CONS

- Abstraction overhead
- Hidden queries

ADVANCED / NEW FEATURES

- Official driver improvements
- Mongoose 8
- Prisma support

12. Array and Embedded Document Queries

DEFINITION

`$elemMatch`, dot notation, array update operators and positional operators.

SYNTAX

```
{ 'addresses.city': 'Chennai', tags: { $elemMatch: ... } }
```

EXAMPLE

```
db.users.find({ 'addresses.city': 'Chennai' })
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Searching inside lists on a card, such as 'any address in Chennai'; array and embedded queries look inside documents.

தமிழ்

அட்டையில் உள்ள பட்டியலுக்குள் 'சென்னையில் ஏதாவது முகவரி' எனத் தேடுவது போல array, embedded queries ஆவணங்களுக்குள் பார்க்கின்றன.

PROS

- Natural nested queries
- `$elemMatch`

CONS

- Complex array queries need care

ADVANCED / NEW FEATURES

- Multikey index improvements

Advanced Level

13. Transactions

DEFINITION

Multi-document ACID transactions on replica sets and sharded clusters.

SYNTAX

```
session.withTransaction(async () => { ... })
```

EXAMPLE

```
const s = client.startSession();
await s.withTransaction(async () => { /* ops with { session: s } */ });
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Transferring money between two folders: both updates succeed or neither; multi-document transactions provide ACID.

தமிழ்

இரு கோப்புகளுக்கு இடையே பணம் மாற்றுவது போல இரு மாற்றங்களும் நடக்க வேண்டும் அல்லது எதுவும் இல்லை; transactions ACID தருகின்றன.

PROS

- ACID across documents
- Works on replica sets and shards

CONS

- Performance cost
- Prefer single-document design

ADVANCED / NEW FEATURES

- Transactions on sharded clusters (4.2+)

14. Replication and Replica Sets

DEFINITION

Primary and secondaries provide high availability and automatic failover; read preferences and write concerns.

SYNTAX

```
rs.initiate()
```

EXAMPLE

```
rs.status()
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A head office with branch copies of the records so work continues if one office closes; replica sets provide failover.

தமிழ்

ஒரு அலுவலகம் மூடப்பட்டாலும் பணி தொடர கிளைகளில் பதிவு நகல்கள் இருப்பது போல replica sets failover தருகின்றன.

PROS

- High availability
- Automatic failover
- Read scaling

CONS

- Replication lag
- More servers to manage

ADVANCED / NEW FEATURES

- Read/write concerns
- Atlas multi-region clusters

15. Sharding

DEFINITION

Horizontal scaling by distributing data across shards using a shard key; choose a key with high cardinality.

SYNTAX

```
sh.shardCollection('db.c', { key: 'hashed' })
```

EXAMPLE

```
sh.shardCollection('shop.orders', { customerId: 'hashed' })
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

When one godown is full, goods are spread across many godowns by a rule; sharding distributes data across servers.

தமிழ்

ஒரு கிடங்கு நிரம்பினால் ஒரு விதிப்படி பல கிடங்குகளுக்குப் பொருட்களைப் பகிர்வது போல sharding தரவை பல சேவையகங்களில் பகிர்கிறது.

PROS

- Horizontal scale
- Large datasets

CONS

- Shard key is hard to change
- Operational complexity

ADVANCED / NEW FEATURES

- Resharding (5.0)
- Config shard / embedded config server (8.0)

16. Security

DEFINITION

Enable authentication, role-based access control, TLS, IP allow lists, field-level encryption.

SYNTAX

```
db.createUser({ roles: [...] })
```

EXAMPLE

```
db.createUser({ user: 'app', pwd: '...', roles: [{ role: 'readWrite', db: 'shop' }] })
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Different keys for different rooms, and CCTV at the door; authentication, roles and encryption protect data.

தமிழ்

வெவ்வேறு அறைகளுக்கு வெவ்வேறு சாவிகள், வாயிலில் CCTV போல authentication, roles, encryption தரவைப் பாதுகாக்கின்றன.

PROS

- RBAC
- TLS and encryption at rest
- Field-level encryption

CONS

- Open by default if misconfigured

ADVANCED / NEW FEATURES

- Queryable Encryption
- Atlas IP access list and private networking

17. Performance Tuning

DEFINITION

Right indexes, covered queries, projections, avoiding unbounded arrays, profiler and `explain('executionStats')`.

SYNTAX

```
db.c.find().explain('executionStats')
```

EXAMPLE

```
db.orders.find({ status: 'paid' }).explain('executionStats')
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A doctor checks the pulse before treatment; explain shows how a query runs so you can fix slowness.

தமிழ்

சிகிச்சைக்கு முன் மருத்துவர் நாடி பார்ப்பது போல explain query எப்படி இயங்குகிறது என்று காட்டி மெதுவை சரிசெய்ய உதவுகிறது.

PROS

- Find slow queries
- Profiler built in

CONS

- Needs interpretation skills

ADVANCED / NEW FEATURES

- Atlas Performance Advisor
- Query shape stats

18. Change Streams and Time Series

DEFINITION

React to data changes in real time; store time-stamped metrics efficiently.

SYNTAX

```
db.c.watch()
```

EXAMPLE

```
db.orders.watch().on('change', c => console.log(c));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A doorbell that rings when a parcel arrives; change streams notify your app when data changes.

தமிழ்

பார்சல் வந்ததும் ஒலிக்கும் அழைப்பு மணி போல change streams தரவு மாறியதும் செயலிக்குத் தெரிவிக்கின்றன.

PROS

- Real-time reactions
- No polling

CONS

- Needs replica set
- Resume token handling

ADVANCED / NEW FEATURES

- Time series collections (5.0)
- Pre/post-images (6.0)

Master Level

19. Atlas Search and Vector Search

DEFINITION

Full-text and semantic search directly in MongoDB Atlas for modern AI apps.

SYNTAX

```
{ $search: { text: { query: 'x', path: 'f' } } }
```

EXAMPLE

```
{ $search: { text: { query: 'laptop', path: 'name' } } }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Finding similar-looking sarees by describing them, not exact words; vector search finds meaning, not just keywords.

தமிழ்

சரியான சொற்கள் இல்லாமல் விவரித்து ஒரே மாதிரியான சேலைகளைத் தேடுவது போல vector search சொற்களை அல்ல, பொருளைத் தேடுகிறது.

PROS

- Search and AI in one database
- No separate search engine

CONS

- Atlas-only features
- Cost at scale

ADVANCED / NEW FEATURES

- Atlas Vector Search
- Hybrid search
- Voyage AI embeddings

20. Backup, Restore and Monitoring

DEFINITION

`mongodump` / `mongorestore`, Atlas backups, monitoring, alerting and capacity planning.

SYNTAX

```
mongodump --uri "$URI"
```

EXAMPLE

```
mongodump --uri "$URI" --out backup/
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Keeping safe duplicate copies and watching the shop cameras; backups and monitoring protect production.

தமிழ்

பாதுகாப்பான நகல்களை வைத்து கடை கேமராக்களைக் கவனிப்பது போல backups, monitoring உற்பத்தியைப் பாதுகாக்கின்றன.

PROS

- Point-in-time restore on Atlas
- Metrics and alerts

CONS

- Self-managed backups need effort

ADVANCED / NEW FEATURES

- Atlas continuous backup
- Ops Manager

21. Production Best Practices

DEFINITION

Connection pooling, schema versioning, indexes review, data lifecycle (TTL, archiving), least privilege.

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Keeping the shop tidy: indexes reviewed, secrets safe, old data archived; production best practices keep systems healthy.

தமிழ்

கடையை ஒழுங்காக வைத்தல்: indexes பரிசீலனை, ரகசியங்கள் பாதுகாப்பு, பழைய தரவு காப்பகம்; production best practices அமைப்புகளை ஆரோக்கியமாக வைக்கின்றன.

PROS

- Stable operations
- Predictable cost

CONS

- Needs discipline

ADVANCED / NEW FEATURES

- TTL indexes
- Data federation / online archive

22. Real Project: Blog / E-commerce Backend

DEFINITION

Model users, products and orders; build APIs with Express + Mongoose; add indexes, aggregation reports and Atlas deployment.

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A blog or online store backend with users, posts/products and orders is the standard practice project.

தமிழ்

பயனர்கள், பதிவுகள்/பொருட்கள், ஆர்டர்கள் கொண்ட blog அல்லது ஆன்லைன் கடை backend ஒரு நிலையான பயிற்சி project.

PROS

- Real-world practice

CONS

- Takes time

ADVANCED / NEW FEATURES

- Use Atlas free tier + Mongoose + Express

Interview Question Bank

Q1. SQL vs NoSQL?
SQL uses fixed relational schemas and joins; NoSQL (like MongoDB) uses flexible documents and scales horizontally.

Q2. What is BSON?
Binary JSON format with extra types (dates, ObjectId) used by MongoDB.

Q3. Embedding vs referencing?
Embed for data read together and bounded size; reference for large, shared or unbounded data.

Q4. What is the aggregation pipeline?
A sequence of stages that transform documents into aggregated results.

Q5. What is a replica set?
A group of mongod instances with one primary and secondaries for high availability.

Q6. What is sharding?
Partitioning data across servers using a shard key for horizontal scaling.

Q7. What is `_id`?
The mandatory unique primary key of every document, ObjectId by default.

Q8. How do indexes work in MongoDB?
B-tree structures that avoid collection scans; compound index order matters.

Q9. Does MongoDB support transactions?
Yes, multi-document ACID transactions since 4.0 (replica sets) and 4.2 (sharded).

Q10. What is Mongoose?
An ODM for Node.js providing schemas, validation and models.

Capstone Project Ideas

1. Blog backend
2. E-commerce catalogue with aggregation reports
3. Chat app storing messages
4. IoT time-series logger
5. Task manager API with Mongoose